

Implement the processing Logic- Runbook

- **Purpose:** Dockerized Application Setup and Execution
- **Author:** Prashant Dixit
- **Date:** July 22, 2024
- **Version:** 1.0

Table of Contents
1. Overview
2. Prerequisites
3. Project Structure
4. Running the Application - Step 1
5. Running the Application - Step 2
6. Accessing the Jupyter Notebook
7. Key Points to Remember



1. Overview

This project focuses on processing and analyzing visual content data using Apache Spark within a Dockerized environment. The application ingests data from JSON files, performs ETL (Extract, Transform, Load) operations, and outputs processed data and metrics. Key technologies and components include:

Technologies Used:

Apache Spark | Docker | Python

Project Setup and Automation:

- The Dockerfile ensures a consistent environment with dependencies specified in requirements.txt`.
- The main script (`visual\_content\_processor.py`) handles ETL operations, transforming and outputting JSON data.
- A Python wrapper script (`run\_build\_and\_run.py`) automates Docker image build and run processes.
- `requirements.txt` lists all necessary external packages.

Deployment and Usage

- The application runs locally or on JupyterHub, supporting both development and collaboration.
- A Jupyter notebook interface enhances accessibility and ease of use.
- Date-stamped output files ensure clear versioning and traceability.
- The Docker image is pushed to Docker Hub for easy team access.

2. Prerequisites

To run the application smoothly, we need the right tools and setup. These prerequisites ensure everything is in place for Docker, Python, and Spark to work seamlessly, preventing issues and making the process easier.

Python Installed: Python should be installed for running local scripts and the wrapper.

[Python Installation Guide](#)

Docker Installed: Ensure Docker is installed and running on your machine.

[Docker Installation Guide](#)

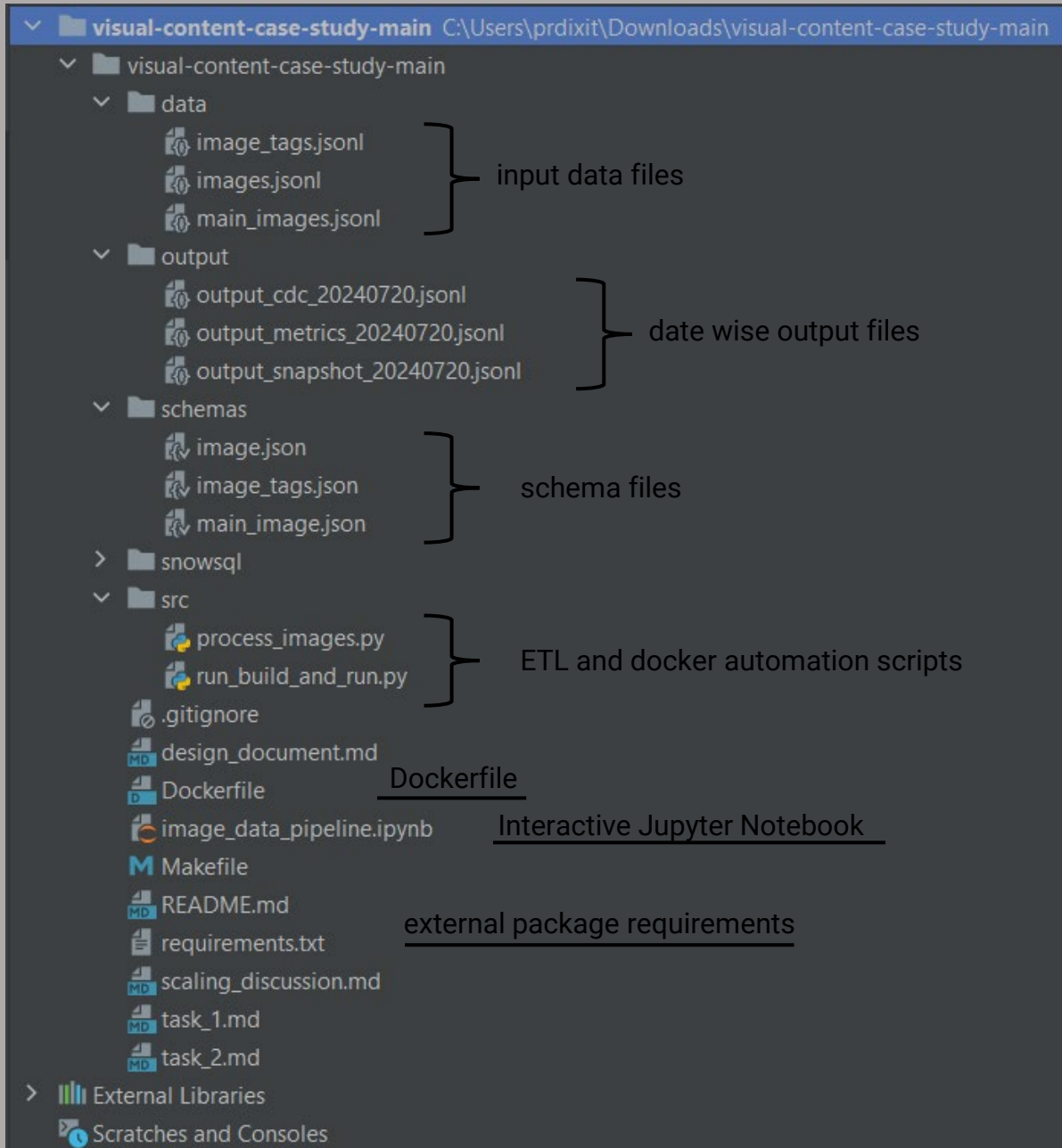
Access to Docker Hub: Ensure you have access to Docker Hub to pull the necessary images.

[Docker Hub](#)

Permissions: Ensure you have the necessary permissions to execute Docker and Python commands, especially in environments with restricted access like corporate machines.

Network Access: Ensure network access for pulling Docker images and any other dependencies.

3. Project Structure



Project structure is organized to segregate files and scripts logically, making it easier to navigate.

- src/: contains the main ETL script.
- data/: stores input JSON files.
- output/: saves generated output files.
- requirements.txt: lists necessary Python packages.
- Dockerfile: defines the Docker image build process.
- image_data_pipeline.ipynb: Jupyter Notebook to run the ETL script and view results interactively.

4. Running the Application - Step 1

To run the application, you can choose to run it locally or use a Docker image.

1. Ensure all dependencies are installed by running:
pip install -r requirements.txt
2. To test locally: directly run the python command below over the terminal:
(venv) PS C:\Users\prdixit\Downloads\visual-content-case-study-main\visual-content-case-study-main>
python src/process_images.py
--images data/images.jsonl --tags data/image_tags.jsonl
--main_images data/main_images.jsonl
--output_cdc output/output_cdc.jsonl
--output_snapshot output/output_snapshot.jsonl
--output_metrics output/output_metrics.jsonl

```
Terminal: Local x build_and_run.sh x build_and_run.sh x + v
(venv) PS C:\Users\prdixit\Downloads\visual-content-case-study-main\visual-content-case-study-main> python src/process_images.py --images data/images.jsonl --tags data/image_tags.jsonl --main_images data/main_images.jsonl --output_cdc output/output_cdc.jsonl --output_snapshot output/output_snapshot.jsonl --output_metrics output/output_metrics.jsonl
24/07/21 15:27:32 WARN Shell: Did not find winutils.exe: java.io.FileNotFoundException: java.io.FileNotFoundException: HADOOP_HOME and hadoop.home.dir are unset. -see https://wiki.apache.org/hadoop/WindowsProblems
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
24/07/21 15:27:32 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
24/07/21 15:27:35 WARN SizeEstimator: Failed to check whether UseCompressedOops is set; assuming yes
Images DataFrame:
+-----+-----+-----+-----+-----+-----+-----+-----+
|cdn_url|created_at|hash|height|hotel_id|image_id|is_active|width|
+-----+-----+-----+-----+-----+-----+-----+-----+
|https://imgcy.trivago.com/partner-images/14/5e/f1828f8199e1b1fbc8aa4a213462586e1950afe08b1ff6f7b1a0cf3a992e|2023-07-19|fffe0458dc3c0000|1365|7009|518367610|true|2048|
|https://imgcy.trivago.com/partner-images/28/ee/0919deb85e9082313200b8a5c7a4727cee2d980d09b5f73720005172b628|2023-07-06|1c3cfe80c3def0f0|533|7001|468437792|true|800|
|https://imgcy.trivago.com/partner-images/fb/82/5fd713ed4b9d85e4cfe3d8f4a4f7b631ec481b0d65b4d718c6aec7d30ab0|2023-09-26|1118183b3e78183f|2362|7009|634688326|true|3543|
|https://imgcy.trivago.com/partner-images/1e/ce/5773e09509dbb7c2870e0e01c0907e0d7e5e7d01c473de515c284713027|2023-09-13|b5100000701d505d12510|7000|150401770|true|17040|
```

5. Running the Application - Step 2

Running Using Docker Image: [Option 1]: Build and Run from PyCharm IDE:

1. Open PyCharm and open the project from the unzipped directory.
2. Build the Docker image using the provided Dockerfile:

<docker build -t visual-content-case-study .>

3. Run the Docker container:

<docker run -p 8888:8888 -v C:/Users/prdixit/Downloads/visual-content-case-study-main/visual-content-case-study-main:/visual-content-case-study-main visual-content-case-study>

4. A link to Jupyter will be displayed in the terminal. Open it in your browser.

5. In Jupyter, open the image_data_pipeline.ipynb notebook and run the cells to trigger the ETL script and view results interactively.

Note: commands are enclosed within <> to assist in copying over, please remove < and > characters before using.

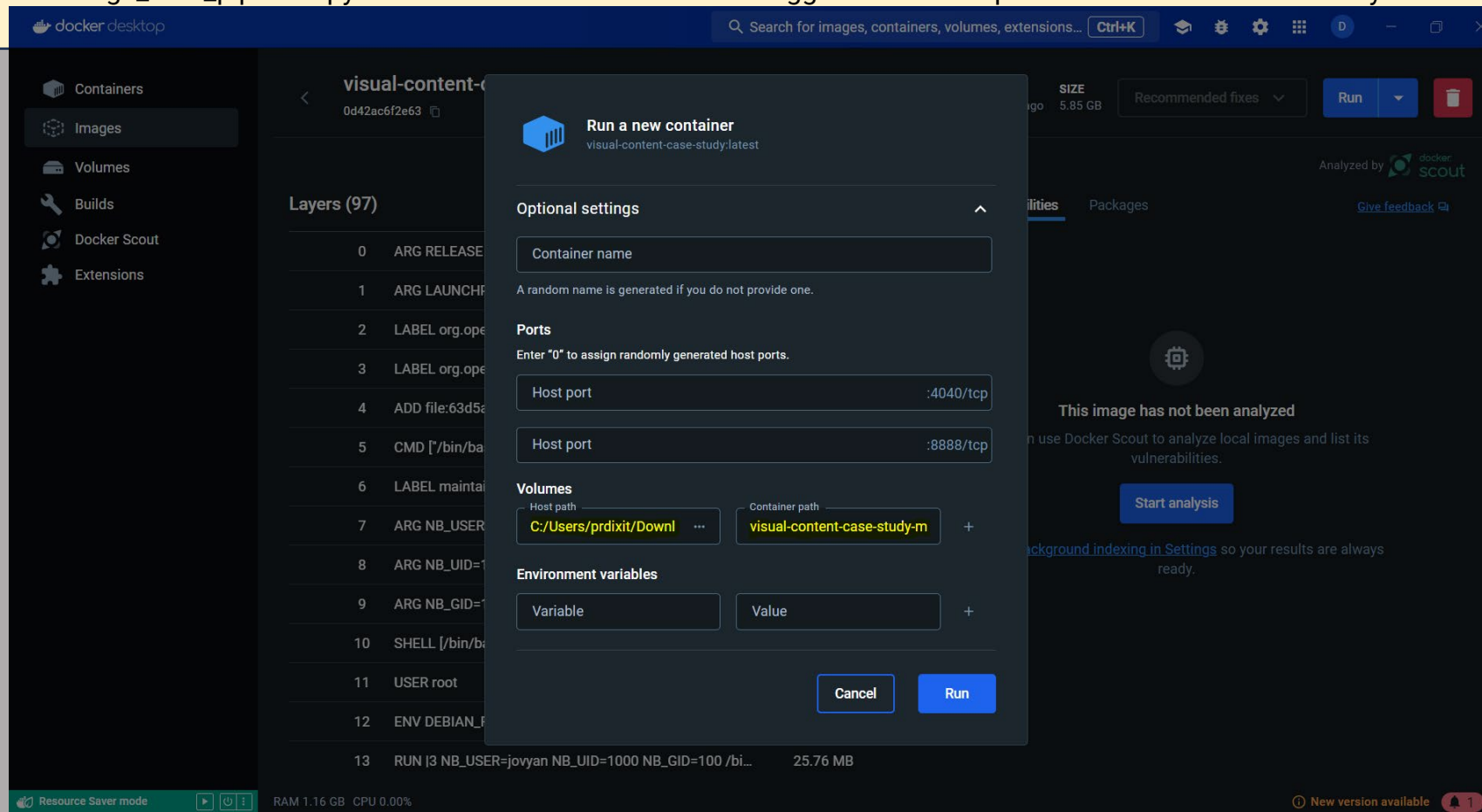
```
Terminal: Local x + v
(venv) PS C:\Users\prdixit\Downloads\visual-content-case-study-main\visual-content-case-study-main> docker build -t visual-content-case-study .
[+] Building 2.2s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 438B
=> [internal] load metadata for docker.io/jupyter/pyspark-notebook:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/4] FROM docker.io/jupyter/pyspark-notebook:latest@sha256:58377aaa152b741e244f201679f96d909a024ea337088cc276b0ee32ab3f076f
=> => resolve docker.io/jupyter/pyspark-notebook:latest@sha256:58377aaa152b741e244f201679f96d909a024ea337088cc276b0ee32ab3f076f
=> [internal] load build context
=> => transferring context: 2.25kB
=> CACHED [2/4] WORKDIR /visual-content-case-study-main
=> CACHED [3/4] COPY . /visual-content-case-study-main
=> CACHED [4/4] RUN pip install -r requirements.txt
=> exporting to image
=> => exporting layers
=> => writing image sha256:0d42ac6f2e63777603578d7cf9fc4a949744dad1ae67744f7a213ad08371aa46
=> => naming to docker.io/library/visual-content-case-study

What's next:
View a summary of image vulnerabilities and recommendations → docker scout quickview
```

.....Running the Application - Step 2

Running Using Docker Image: [Option 2]: Pull and Run from Docker Desktop:

1. Open Docker Desktop.
2. Pull the Docker image using the command:
`<docker pull docker1app/visual-content-case-study:v1.0>`
3. Run the Docker container:
`docker run -p 8888:8888 your_dockerhub_username/visual-content-case-study`
4. A link to Jupyter will be displayed in the terminal. Open it in your browser.
5. In Jupyter, open the image_data_pipeline.ipynb notebook and run the cells to trigger the ETL script and view results interactively.



6. Accessing the Jupyter Notebook

Jupyter Notebook: An interactive platform to run Python code, visualize data, and document workflows.

Server Link: When the Docker container is started, it provides a link to the Jupyter server, which you can open in your browser.

Volume Mapping (-v option): This Docker option maps a directory on your local machine to a directory inside the container, ensuring that any changes you make in the notebook are saved locally. For example:

```
docker run -p 8888:8888 -v C:/Users/prdixit/Downloads/visual-content-case-study-main/visual-content-case-study-main:/visual-content-case-study-main visual-content-case-study
```

This command maps `C:/Users/prdixit/Downloads/visual-content-case-study-main/visual-content-case-study-main` on your machine to `/visual-content-case-study-main` inside the Docker container, allowing persistent storage of your work.

```
I 2024-07-21 11:12:29.305 ServerApp] jupyter_server_terminals | extension was successfully loaded.
I 2024-07-21 11:12:29.322 LabApp] JupyterLab extension loaded from /opt/conda/lib/python3.11/site-packages/jupyterlab
I 2024-07-21 11:12:29.323 LabApp] JupyterLab application directory is /opt/conda/share/jupyter/lab
I 2024-07-21 11:12:29.323 LabApp] Extension Manager is 'pypi'.
I 2024-07-21 11:12:29.326 ServerApp] jupyterlab | extension was successfully loaded.
I 2024-07-21 11:12:29.330 ServerApp] jupyterlab_git | extension was successfully loaded.
I 2024-07-21 11:12:29.338 ServerApp] nbclassic | extension was successfully loaded.
I 2024-07-21 11:12:29.447 ServerApp] nbdime | extension was successfully loaded.
I 2024-07-21 11:12:29.450 ServerApp] notebook | extension was successfully loaded.
I 2024-07-21 11:12:29.450 ServerApp] Serving notebooks from local directory: /visual-content-case-study-main
I 2024-07-21 11:12:29.450 ServerApp] Jupyter Server 2.8.0 is running at:
I 2024-07-21 11:12:29.450 ServerApp] http://4657b39396d3:8888/lab?token=7e6e4cd31012ebdd8a4f234d4ed958d34e9871a3453a3727
I 2024-07-21 11:12:29.450 ServerApp] http://127.0.0.1:8888/lab?token=7e6e4cd31012ebdd8a4f234d4ed958d34e9871a3453a3727
I 2024-07-21 11:12:29.450 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
C 2024-07-21 11:12:29.453 ServerApp]
```



To access the server, open this file in a browser:

<file:///home/jovyan/.local/share/jupyter/runtime/jpserver-7-open.html>

Or copy and paste one of these URLs:

<http://4657b39396d3:8888/lab?token=7e6e4cd31012ebdd8a4f234d4ed958d34e9871a3453a3727>

<http://127.0.0.1:8888/lab?token=7e6e4cd31012ebdd8a4f234d4ed958d34e9871a3453a3727>

```
I 2024-07-21 11:12:30.154 ServerApp] Skipped non-installed server(s): bash-language-server, dockerfile-language-server-nodejs, javascript-typescript-langserver, jedi-language-server,
ulia-language-server, pyright, python-language-server, python-lsp-server, r-languageserver, sql-language-server, texlab, typescript-language-server, unified-language-server, vscode-cs
-language-server-bin, vscode-html-language-server-bin, vscode-json-language-server-bin, yaml-language-server
```

127.0.0.1:8888/lab/tree/image_data_pipeline.ipynb

File Edit View Run Kernel Tabs Settings Help

Filter files by name

Name Last Modified

- data 5 days ago
- output 14 minutes ago
- schemas 3 days ago
- snowsql 3 days ago
- src 2 hours ago
- design_document.md 9 days ago
- Dockerfile 5 minutes ago
- image_data_pipeline.ipynb 13 minutes ago**
- Makefile yesterday
- README.md 9 days ago
- requirements.txt yesterday
- scaling_discussion.md 9 days ago
- task_1.md 9 days ago
- task_2.md 9 days ago

image_data_pipeline.ipynb

```
# Construct the command
command = [
    'python', '/visual-content-case-study-main/src/process_images.py',
    '--images', images_path,
    '--tags', tags_path,
    '--main_images', main_images_path,
    '--output_cdc', output_cdc_path,
    '--output_snapshot', output_snapshot_path,
    '--output_metrics', output_metrics_path
]

try:
    # Run the command and capture output and errors
    result = subprocess.run(command, check=True, stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True)
    print("Output:\n", result.stdout)
    print("Errors:\n", result.stderr)
except subprocess.CalledProcessError as e:
    print("Command failed with error code", e.returncode)
    print("Output:\n", e.stdout)
    print("Errors:\n", e.stderr)

print("Script execution [COMPLETED] | Output Folder: [output]")
```

Output:

Images DataFrame:

cdn_url	created_at	hash	height	hotel_id	image_id	is_active	width
https://imgcy.trivago.com/partner-images/14/5e/f1828f8199e1b1fbc8aa4a213462586e1950afe08b1ff6f7b1a0cf3a992e	2023-07-19	fffe0458dc3c0000	1365	7009	518	367610	2048
https://imgcy.trivago.com/partner-images/28/ee/0919deb85e9082313200b8a5c7a4727cee2d980d09b5f73720005172b628	2023-07-06	1c3cfe80c3def0f0	533	7001	468	437792	800

task2 - PowerPoint

Mode: Command Ln 24, Col 2 image_data_pipeline.ipynb

- In the Jupyter interface, navigate to the `image\_data\_pipeline.ipynb` notebook. Run the cells in the notebook to execute the ETL script and view the results interactively.

Thank You